CO1 AT3

Annexure A

PSEUDO CODE WRITING

Q1. Given pseudocode:

```
#define DEBUG 0

IF DEBUG
  PRINT intermediate_value
END IF
```

- Identify error in directive usage
- Correct the logic

Q2. #define PI _____

```
DECLARE radius AS float
DECLARE area AS float

INPUT radius

area = _____

PRINT area
```

Fill:

- Constant
- Formula

Q3. #define RATE 1.5

```
DECLARE x AS int = 10
DECLARE result AS float

result = x * RATE

PRINT result
```

Questions:

- Output?
- Why is the result float?

Q4. #define MAX 1000

```
DECLARE temp AS int
INPUT temp

IF temp > MAX
  PRINT "Valid"
ELSE
  PRINT "Invalid"
```

- Identify logical error
- Suggest correct condition

Q5. Given unordered pseudocode steps:

- DEFINE conversion rate
- PRINT result
- INPUT amount
- CALCULATE converted value

Arrange in correct order

Q6. #define MIN 10
#define MAX 50

```
DECLARE value AS int
INPUT value

_____ (value >= MIN AND value <= MAX)
PRINT "Within range"

_____
PRINT "Out of range"
```

Fill missing control structures

Q7. A billing system includes optional features such as discount calculation and tax computation. Depending on the business requirement, these features may be enabled or disabled at compile time. The program should calculate the total bill amount based on input price and quantity. If discount and tax features are enabled, they should be applied using predefined constants; otherwise, they should be skipped. The system must ensure that floating-point precision is maintained during calculations.

Write pseudocode using preprocessor directives to enable/disable features, define constants for discount and tax rates, and use appropriate data types.

Q8. #define TAX 5%

DECLARE price AS int
DECLARE total AS int

total = price + (price * TAX)

PRINT total

- Identify errors
- Correct data types and constant definition

9. A temperature conversion program supports multiple modes such as Celsius to Fahrenheit and Fahrenheit to Celsius. The mode of operation should be selected at compile time using preprocessor directives. The program should use appropriate floating-point data types to ensure accuracy and define conversion formulas using symbolic constants wherever applicable. The final output should be displayed with two decimal precision.
Write pseudocode using preprocessor directives for mode selection, constants for conversion formulas, and proper data types.

10. #define SIZE 5

DECLARE arr[SIZE]
INPUT elements

- Extend pseudocode to:
  - Find sum
  - Compute average

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

1. Identify error in directive usage.

Error :

* #define DEBUG 0 is a C preprocessor directive.
* If DEBUG should Compare the value.

Corrected logic :

```
START
SET DEBUG = 0
IF DEBUG = 1 THEN
   PRINT intermediate_value
END IF
STOP
```

2. Fill Constant, Formula

```
START
SET PI = 3.14
DECLARE RADIUS AS FLOAT
DECLARE area AS FLOAT
INPUT radius
area = PI x radius x radius
PRINT area
STOP
```

Constant → 3.14
Formula → PI x radius x radius

(3). #define RATE 1.5
DECLARE X AS int = 10
DECLARE result AS float
result = X * RATE
PRINT result

Output:

15.0

Reason:

The result is FLOAT because RATE = 1.5 is a floating-point constant. Multiplying an integer by a float gives a float result.

(4) Logical Error: The condition is incorrect because values greater than MAX should be invalid.

Correct Condition:

```
START
SET MAX = 1000
DECLARE temp AS INT
INPUT temp
IF temp <= MAX THEN
   PRINT "Valid"
ELSE
      PRINT "Invalid"
END IF
STOP
```

(5)

Correct Order:

1. DEFINE Conversion rate.
2. INPUT amount.
3. CALCULATE Converted value.
4. PRINT result.

(6) Fill missing Control Structures

```
START
SET MIN = 10
SET MAX = 50
DECLARE value AS INT
INPUT value
IF (value >= MIN AND value <= MAX) THEN
    PRINT "within range"
ELSE
    PRINT "Out of range"
END IF
STOP
```

(7) Pseudocode

```
START
#define DISCOUNT_ENABLED 1
#define TAX_ENABLED 1
#define DISCOUNT_RATE 0.10
#define TAX_RATE 0.05
DECLARE Price AS FLOAT
DECLARE quantity AS INT
DECLARE total AS FLOAT

INPUT Price
INPUT quantity

total = Price * quantity

IF DISCOUNT_ENABLED == 1 THEN
    total = total - (total * DISCOUNT_RATE)
END IF

IF TAX_ENABLED == 1 THEN
    total = total + (total * TAX_RATE)
END IF

PRINT total

STOP
```

(8) Errors:

* #define TAX 5% is incorrect.
* Price and total should be Float.

Pseudocode:

START
SET TAX = 0.05
DECLARE price AS FLOAT
DECLARE total AS FLOAT
INPUT price
total = price + (price x TAX)
PRINT total
STOP

(9) Pseudocode:

START
#define C_TO_F 1
#define F_TO_C 0
#define FACTOR 1.8
#define OFFSET 32
DECLARE temperature AS FLOAT
DECLARE result AS FLOAT

INPUT temperature

IF C_TO_F == 1 THEN
    result = (temperature * FACTOR) + OFFSET
ELSE IF F_TO_C == 1 THEN
    result = (temperature - OFFSET) / FACTOR
END IF
PRINT result (2 decimal places)
STOP

(10)

```
START
SET SIZE = 5
DECLARE arr[SIZE] AS INT
DECLARE i AS INT
DECLARE sum AS INT = 0
DECLARE average AS FLOAT

FOR i = 0 TO SIZE - 1
    INPUT arr[i]
    sum = sum + arr[i]
END FOR
average = sum / SIZE
PRINT "SUM", sum
PRINT "Average=", average
STOP
```